11강

프로그래밍 언어론

# 수식

컴퓨터과학과 김진욱교수

# 학습목차




1 수식의 개요

2 수식과 문장의 차이

3 수식과 연산자

# 01

# 수식의 개요

# 1. 수식의 개요

- 수식(expression)
  - 값을 나타내는 표현
    - 값: 정수, 실수, 문자, 문자열, 배열 표현(C, C++), 리스트 표현(Python) 등

```
2
```

```
-5.3
```

  - 연산자가 피연산자에 적용된 형태도 가능
    - 피연산자: 연산이 적용될 값
      - 데이터 표현, 변수
    - 연산자: 피연산자에 적용되어 연산을 수행하는 함수
      - 기본 연산자, 함수

```
'abc'
```

```
[2, 3, 1]
```

```
i + 3
```

# 1. 수식의 개요

## 수식(expression)

- 피연산자와 연산자가 모두 포함될 필요는 없음




```
i + 3
```
➡
```
4
```

```
-5.3
```
➡
```
-5.3
```

```
“Good” + “ morning”
```
➡
```
“Good morning”
```

# 1. 수식의 개요

## 수식(expression)

- 피연산자와 연산자가 모두 포함될 필요는 없음

```
int f(void) {
  return 100;
}
```

```
f()
```



```
100
```

02

# 수식과 문장의 차이

# 2. 수식과 문장의 차이

- 수식
  - 값을 나타내는 표현

- 문장
  - 처리를 나타내는 표현
    - 처리: 수식을 계산, 프로그램의 수행 흐름 바꾸기, 값을 화면에 출력 등

- 예: Python

```
3 + 2
```

수식이면서 문장

```
if k > 1:
  k = 1
```

문장

# 2. 수식과 문장의 차이

## C, C++

- 수식의 끝에 쌍반점(;)을 찍으면 문장이 됨

수식

```
i = 3 + 2
```

→

```
5
```

문장

```
i = 3 + 2;
```

수식

```
printf("Hi")
```

→

```
2
```

문장

```
printf("Hi");
```

# 2. 수식과 문장의 차이

- C, C++
  - 문장의 끝에서 쌍반점을 뺀다고 항상 수식이 되진 않음

문장

```
if (a > 0)
  printf("positive\n");
```

수식

```
if (a > 0)
  printf("positive\n")
```

# 03

# 수식과 연산자

# 3. 수식과 연산자

- 수식에 사용되는 기본 연산자
  - 산술 연산자
  - 관계 연산자
  - 논리 연산자
  - 그 외 연산자

# 3. 수식과 연산자

## 산술 연산자

- 기본적으로 사칙연산자 포함
  - 덧셈 연산자, 뺄셈 연산자, 곱셈 연산자, 나눗셈 연산자

| 연산자 | 의미 |
|---|---|
| + | 덧셈 |
| - | 뺄셈 |
| * | 곱셈 |
| / | 나눗셈 |

# 3. 수식과 연산자

## 산술 연산자

- 언어에 따라 추가적인 산술 연산자 포함
  - 나머지 연산자, 거듭제곱 연산자 등

### C, C++

| 연산자 | 의미 |
|---|---|
| % | 나머지 |
| ++(전위) | 1 증가 |
| --(전위) | 1 감소 |
| ++(후위) | 값 사용 후 1 증가 |
| --(후위) | 값 사용 후 1 감소 |
| +(부호) | 부호 유지 |
| -(부호) | 부호 반대로 |

### Python

| 연산자 | 의미 |
|---|---|
| // | 몫 |
| % | 나머지 |
| ** | 거듭제곱 |
| +(부호) | 부호 유지 |
| -(부호) | 부호 반대로 |

# 3. 수식과 연산자

## 산술 연산자

- 피연산자 개수에 따른 연산자 분류
  - 단항 연산자: 1개의 피연산자에 적용되는 연산자
  - 이항 연산자: 2개의 피연산자에 적용되는 연산자

### C, C++ 단항 연산자

| 연산자 | 의미 | 연산자 대비 피연산자 위치 |
|---|---|---|
| ++(전위) | 1 증가 | 우 |
| --(전위) | 1 감소 | 우 |
| ++(후위) | 값 사용 후 1 증가 | 좌 |
| --(후위) | 값 사용 후 1 감소 | 좌 |
| +(부호) | 부호 유지 | 우 |
| -(부호) | 부호 반대로 | 우 |

```
++i
```

```
i++
```

```
+i
```

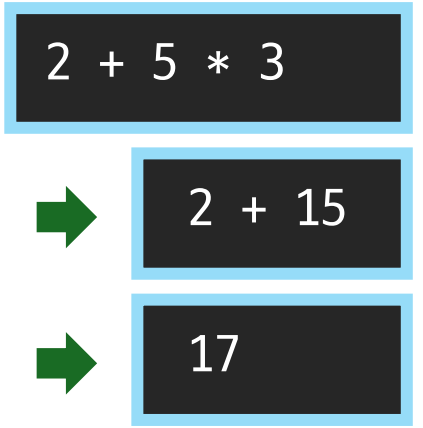
# 3. 수식과 연산자

## 산술 연산자

- 우선순위
  - 하나의 수식에 여러 연산자가 사용되는 경우 우선순위에 따라 피연산자를 모음

C, C++

| 연산자 | 우선순위 | 결합방향 |
|---|---|---|
| ++(후위), --(후위) | 1 | 좌결합 |
| ++(전위), --(전위), +(부호), -(부호) | 2 | 우결합 |
| *, /, % | 3 | 좌결합 |
| +, - | 4 | 좌결합 |
| >>, << | 5 | 좌결합 |



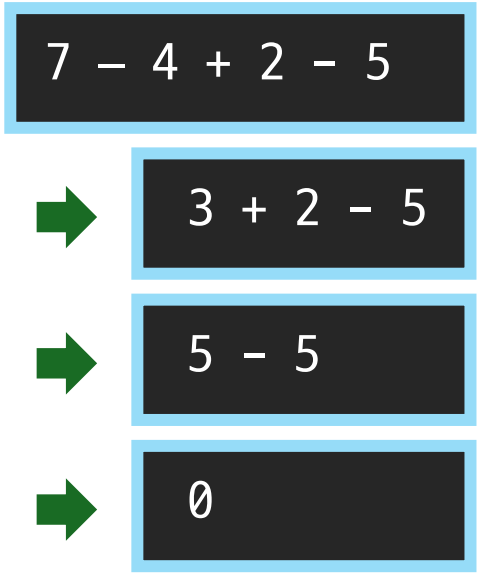
# 3. 수식과 연산자

## 산술 연산자

- 우선순위
  - 우선순위가 같은 연산자들이 섞여 있는 경우 결합방향에 맞춰 피연산자를 모음

C, C++

| 연산자 | 우선순위 | 결합방향 |
|---|---|---|
| ++(후위), --(후위) | 1 | 좌결합 |
| ++(전위), --(전위), +(부호), -(부호) | 2 | 우결합 |
| *, /, % | 3 | 좌결합 |
| +, - | 4 | 좌결합 |
| >>, << | 5 | 좌결합 |



# 3. 수식과 연산자

## 산술 연산자

- 우선순위
  - 우선순위가 같은 연산자들이 섞여 있는 경우 결합방향에 맞춰 피연산자를 모음




C, C++

| 연산자 | 우선순위 | 결합방향 |
|---|---|---|
| ++(후위), --(후위) | 1 | 좌결합 |
| ++(전위), --(전위), +(부호), -(부호) | 2 | 우결합 |
| *, /, % | 3 | 좌결합 |
| +, - | 4 | 좌결합 |
| >>, << | 5 | 좌결합 |

```
- ++ i
```

➡ -3

# 3. 수식과 연산자

## 산술 연산자

- 우선순위
  - 하나의 수식에 여러 연산자가 사용되는 경우 우선순위에 따라 피연산자를 모음




```
- i ++
```

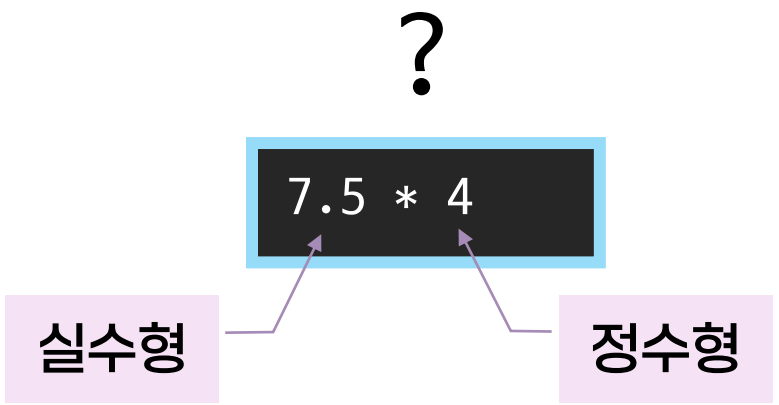
```
-2
```

C, C++

| 연산자 | 우선순위 | 결합방향 |
|---|---|---|
| ++(후위), --(후위) | 1 | 좌결합 |
| ++(전위), --(전위), +(부호), -(부호) | 2 | 우결합 |
| *, /, % | 3 | 좌결합 |
| +, - | 4 | 좌결합 |
| >>, << | 5 | 좌결합 |

# 3. 수식과 연산자

- 산술 연산자
  - 타입 변환
    - 주어진 타입의 값을 다른 타입의 값으로 변환
    - 피연산자들의 타입이 서로 일치하지 않는 경우 필요
    - 종류: 묵시적 타입 변환, 명시적 타입 변환



# 3. 수식과 연산자

## ❑ 산술 연산자

### 묵시적 타입 변환

- 프로그래밍 언어의 정책에 따라 컴파일러에 의해 자동으로 변환
- 대부분 언어는 데이터 집합이 커지는 방향의 타입으로 확대변환

```
7.5 * 4
```

➡

```
7.5 * 4.0
```

➡

```
30.0
```

# 3. 수식과 연산자

## ❑ 산술 연산자

### 명시적 타입 변환(타입 캐스팅)

- 프로그래머가 명시한 타입으로 변환
- 확대변환 및 축소변환 모두 가능

```
7.5 * (double)4   ➡  7.5 * 4.0   ➡  30.0
```

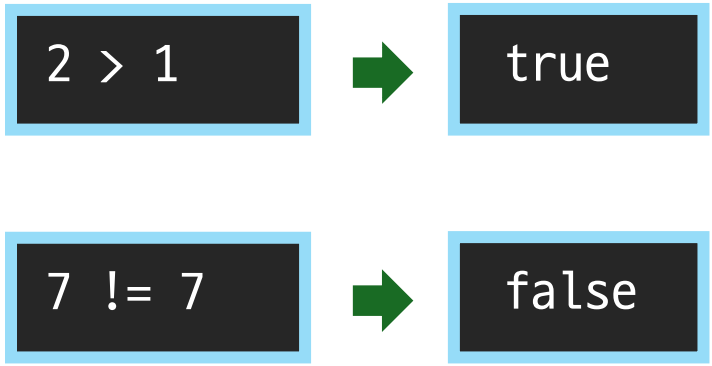
```
(int)7.5 * 4      ➡  7 * 4       ➡  28
```

# 3. 수식과 연산자

## 관계 연산자

- 2개의 피연산자를 서로 비교하여 둘 사이의 관계가 관계 연산자가 나타내는 의미와 일치하는지 여부 확인
- 결과 값: 참(true) 또는 거짓(false)

| 연산자 | 의미 |
|---|---|
| > | 보다 크다 |
| >= | 보다 크거나 같다 |
| == | 같다 |
| <= | 보다 작거나 같다 |
| < | 보다 작다 |
| != | 다르다 |



# 3. 수식과 연산자

## 관계 연산자

- 우선순위
  - 산술 연산자보다 우선순위가 낮음

C, C++

| 연산자 | 우선순위 | 결합방향 |
|---|---|---|
| ++, --(후) | 1 | 좌결합 |
| ++,--(전),+,-(부호) | 2 | 우결합 |
| *, /, % | 3 | 좌결합 |
| +, - | 4 | 좌결합 |
| >>, << | 5 | 좌결합 |
| >, >=, <=, < | 6 | 좌결합 |
| ==, != | 7 | 좌결합 |

```
3 > 2 == 7 - 4 > 1
```

➡
```
3 > 2 == 3 > 1
```

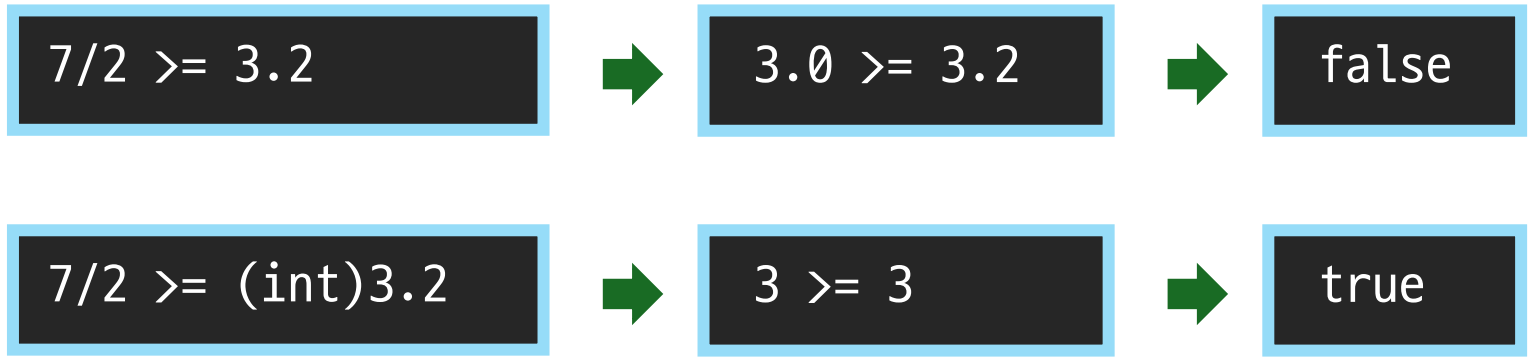
➡
```
true == true
```

➡
```
true
```

# 3. 수식과 연산자

## 관계 연산자

- 타입 변환
  - 묵시적 타입 변환이나 명시적 타입 변환 가능



# 3. 수식과 연산자

## 논리 연산자

- 논리곱 연산자, 논리합 연산자, 논리부정 연산자 등
- 피연산자 및 결과 값: 참(true) 또는 거짓(false)

| 연산자 | 의미 |
|---|---|
| && | 논리곱 |
| \|\| | 논리합 |
| ! | 논리부정 |

```
true && false  ➡  false
```

```
! true  ➡  false
```

# 3. 수식과 연산자

## 논리 연산자

- 우선순위

C, C++

| 연산자 | 우선순위 | 결합방향 |
|---|---|---|
| ++, --(후) | 1 | 좌결합 |
| ++,--(전),+,-(부호), ! | 2 | 우결합 |
| *, /, % | 3 | 좌결합 |
| +, - | 4 | 좌결합 |
| >>, << | 5 | 좌결합 |
| >, >=, <=, < | 6 | 좌결합 |
| ==, != | 7 | 좌결합 |
| && | 8 | 좌결합 |
| \|\| | 9 | 좌결합 |

```
3 > 2 || 5 >= 2 && 3 <= 2
```

➡
```
true || true && false
```

➡
```
true || false
```

➡
```
true
```

# 3. 수식과 연산자

## 논리 연산자

- 단락 회로 계산(short-circuit evaluation)
  - 단축 평가라고도 함
  - 일부 피연산자를 계산하지 않고 수식 값을 결정하는 계산법

```
true || ...
```
➡
```
true
```

```
false && ...
```
➡
```
false
```

```
3 > 2 || 5 >= 2 && 3 <= 2
```
➡
```
true || 5 >= 2 && 3 <= 2
```
➡
```
true
```

# 3. 수식과 연산자

## 그 외 연산자

- 프로그래밍 언어에 따라 더 많은 기본 연산자가 제공되기도 함

C, C++

| 연산자 | 의미 | 피연산자 개수 |
|---|---|---|
| & | 비트별 논리곱 | 2 |
| ^ | 비트별 배타적 논리합 | 2 |
| \| | 비트별 논리합 | 2 |
| ~ | 비트별 논리부정 | 1 |
| ? : | 조건 연산자 | 3 |
| >> | 오른쪽 비트 이동 | 2 |
| << | 왼쪽 비트 이동 | 2 |

# 정리하기

- 수식이란 값을 나타내는 표현으로 데이터 표현 그대로일 수도, 변수일 수도, 피연산자에 연산자가 적용된 형태일 수 있음
- 수식은 값을 나타내는 표현이며 문장은 처리를 나타내는 표현임
- 산술 연산자에는 사칙 연산자 외에도 언어에 따라 나머지 연산자, 부호 연산자 등이 포함됨
- 타입 변환은 주어진 타입의 값을 다른 타입의 값으로 변환하는 것으로 묵시적 타입 변환과 명시적 타입 변환이 있음
- 관계 연산은 2개의 피연산자를 서로 비교하여 둘 사이의 관계가 관계 연산자와 일치하는지 여부를 확인함
- 논리 연산은 단락 회로 계산을 통해 모든 논리 연산을 수행하지 않고도 수식의 값을 얻을 수 있음

12강

다음시간안내 ▶▶▶

# 문장